

LAPORAN ANALISIS KODE MACHINE LEARNING

Supervised Learning



Disusun Oleh:

Nama : Judika Ebenezer Sianturi

NPM : G1A024031

Kelas : A

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU
2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Machine Learning merupakan cabang kecerdasan buatan yang memungkinkan komputer belajar dari data tanpa harus diprogram secara eksplisit. Notebook praktikum ini mengacu pada Bab 2 buku "Introduction to Machine Learning with Python" karya Andreas C. Müller dan Sarah Guido, yang membahas berbagai algoritma supervised learning sebagai fondasi penting dalam praktik machine learning.

Supervised learning adalah jenis pembelajaran mesin di mana model dilatih menggunakan data berlabel untuk memprediksi output pada data baru. Praktikum ini memperkenalkan berbagai algoritma klasifikasi dan regresi, mulai dari yang sederhana seperti k-Nearest Neighbors (k-NN) hingga yang kompleks seperti Neural Networks (MLP), serta teknik-teknik seperti Naive Bayes, Decision Trees, Random Forest, Gradient Boosting, dan Support Vector Machines.

Pemahaman mendalam tentang setiap algoritma sangat penting karena masing-masing memiliki kekuatan dan keterbatasan yang berbeda. Faktor-faktor seperti jumlah data, jumlah fitur, jenis masalah (klasifikasi atau regresi), dan kebutuhan interpretabilitas model memengaruhi pilihan algoritma yang tepat.

Dalam laporan ini, dilakukan implementasi dan analisis berbagai algoritma supervised learning menggunakan Python dan pustaka scikit-learn beserta mglearn. Berbagai dataset sintetis (forge, wave, two_moons, make_blobs, make_circles) dan dataset nyata (Breast Cancer Wisconsin, Boston Housing, Iris) digunakan untuk mengevaluasi performa masing-masing algoritma. Setiap algoritma dievaluasi pada kemampuan generalisasi, risiko overfitting/underfitting, serta pengaruh hyperparameter terhadap performa model.

1.2 Tujuan

- Memahami cara kerja dan implementasi berbagai algoritma supervised learning menggunakan Python.
- Menganalisis performa model k-NN, Linear Models, Naive Bayes, Decision Trees, Random Forest, Gradient Boosting, SVM, dan Neural Networks pada berbagai dataset.
- Memahami pengaruh hyperparameter terhadap kompleksitas model dan risiko overfitting/underfitting.
- Memahami kapan dan mengapa penskalaan fitur diperlukan untuk algoritma tertentu.

- Menginterpretasikan output prediksi model termasuk decision function dan probabilitas prediksi.
- Mengembangkan kemampuan praktis dalam implementasi machine learning menggunakan scikit-learn.

BAB II

METODOLOGI DAN OUTPUT EKSPERIMEN

2.1 Persiapan dan Dataset

Sebelum masuk ke eksperimen algoritma, langkah pertama adalah menyiapkan pustaka pendukung, yaitu `mglearn` (paket pendamping buku yang menyediakan dataset sintetis dan fungsi visualisasi), `matplotlib`, dan `numpy`.

Dua dataset sintetis digunakan sebagai contoh dasar. Dataset `forge` adalah dataset klasifikasi biner kecil (26 sampel, 2 fitur) yang divisualisasikan sebagai scatter plot dua kelas (Class 0 dan Class 1). Ukurannya yang kecil membuatnya mudah digunakan untuk menggambarkan bagaimana suatu algoritma klasifikasi menarik batas keputusan. Dataset `wave` adalah dataset regresi sintetis (40 sampel, 1 fitur kontinu) yang digunakan untuk mendemonstrasikan algoritma regresi, karena targetnya berupa nilai kontinu, bukan label kelas.

Kedua dataset ini sengaja dibuat sederhana (berdimensi rendah) agar batas keputusan atau garis regresi yang dihasilkan model dapat divisualisasikan secara langsung dalam bidang dua dimensi.

2.2 Dataset Nyata: Breast Cancer dan Boston Housing

Untuk menguji algoritma pada data yang lebih realistis, digunakan dua dataset nyata. Breast Cancer Wisconsin dimuat melalui `load_breast_cancer()` dari `scikit-learn`, berisi 569 sampel dengan 30 fitur numerik hasil pengukuran sel tumor. Targetnya biner: malignant (ganas) dan benign (jinak). Dataset ini menjadi dataset utama yang dipakai berulang kali di sepanjang praktikum untuk membandingkan performa antar algoritma klasifikasi.

Boston Housing tidak lagi tersedia langsung di `scikit-learn` versi 1.2 ke atas karena pertimbangan etis pada salah satu fiturnya, sehingga dataset ini dimuat melalui `mglearn.datasets.load_extended_boston()`. Versi `extended` menghasilkan 104 fitur dari 13 fitur asli melalui `feature engineering` (perkalian antar pasangan fitur), yang dipakai untuk menunjukkan efek `overfitting` pada model linear berdimensi tinggi.

2.3 Algoritma k-Nearest Neighbors (k-NN)

k-Nearest Neighbors (k-NN) adalah algoritma supervised learning yang memprediksi label kelas berdasarkan mayoritas label dari k tetangga terdekat pada ruang fitur. Algoritma ini tidak membentuk model secara eksplisit (lazy learning) - ia menyimpan seluruh data latih dan melakukan perhitungan saat prediksi.

Model `KNeighborsClassifier(n_neighbors=3)` dilatih pada dataset `forge` yang telah dibagi menjadi data latih dan data uji, lalu dievaluasi akurasi. Percobaan lanjutan memvariasikan

`n_neighbors` menjadi 1, 3, dan 9 untuk melihat bagaimana batas keputusan berubah: semakin kecil nilai `k`, batas keputusan semakin bergerigi dan mengikuti data latih secara detail (berisiko *overfitting*), sedangkan semakin besar nilai `k`, batas keputusan semakin halus dan sederhana (berisiko *underfitting* jika `k` terlalu besar). Pola ini divalidasi pada dataset Breast Cancer dengan membandingkan akurasi training dan test untuk `n_neighbors` dari 1 hingga 10, memperlihatkan *sweet spot* kompleksitas model pada nilai `k` menengah.

Untuk kasus regresi, `KNeighborsRegressor` diuji pada dataset *wave*. Alih-alih voting mayoritas, prediksi regresi dihitung sebagai rata-rata nilai target dari `k` tetangga terdekat. Dengan `k=1`, prediksi berfluktuasi tajam mengikuti setiap titik data (*overfitting*). Dengan `k=3` dan `k=9`, prediksi menjadi semakin halus dan mengikuti pola umum data.

`k-NN` mudah dipahami dan sering memberikan performa yang layak tanpa banyak penyesuaian, namun menjadi lambat ketika jumlah sampel besar dan performanya buruk pada dataset dengan banyak fitur, sehingga kurang cocok untuk dataset berdimensi tinggi.

2.4 Model Linear (Linear Models)

Model linear membuat prediksi menggunakan kombinasi linear dari fitur-fitur input. Untuk klasifikasi, model linear mencari garis (atau *hyperplane* dalam dimensi tinggi) yang memisahkan kedua kelas. Untuk regresi, model mencocokkan sebuah garis lurus (atau *hyperplane*) pada data.

`LinearRegression` dilatih pada dataset *wave* (fitur tunggal) maupun *extended Boston* (104 fitur). Pada dataset *wave*, skor training dan test relatif rendah namun berdekatan, indikasi model *underfitting* karena hanya memiliki satu fitur. Sebaliknya, pada *extended Boston* yang berdimensi tinggi, skor training jauh lebih tinggi daripada skor test, indikasi *overfitting* klasik akibat model yang terlalu fleksibel dibanding jumlah data.

Ridge Regression menambahkan regularisasi L2 pada *Linear Regression* untuk mencegah *overfitting*. Parameter `alpha` mengontrol kekuatan regularisasi: `alpha` besar menghasilkan koefisien mendekati nol (model sederhana/*underfitting*), `alpha` kecil memberikan lebih banyak kebebasan pada koefisien (model kompleks, risiko *overfitting*). Kurva jumlah data latih terhadap performa juga menunjukkan bahwa seiring bertambahnya data, performa *Ridge* dan *Linear Regression* cenderung menyatu.

Lasso Regression menggunakan regularisasi L1 yang dapat membuat sebagian koefisien menjadi tepat nol, sehingga secara otomatis melakukan seleksi fitur. Semakin kecil `alpha`, semakin banyak fitur yang digunakan model; `alpha` besar menghasilkan model yang sangat sederhana dengan sedikit fitur aktif, berguna ketika interpretabilitas menjadi prioritas.

Untuk klasifikasi, LogisticRegression dan LinearSVC dibandingkan pada dataset forge, keduanya menghasilkan batas keputusan berupa garis lurus meskipun kriteria optimasinya berbeda. Parameter regularisasi C berperan seperti $1/\alpha$ pada model regresi: C kecil menghasilkan regularisasi kuat (model sederhana), C besar membuat model berusaha mencocokkan data latih sedekat mungkin (risiko overfitting). Pada dataset Breast Cancer, perbandingan $C=1$, $C=100$, dan $C=0,01$ pada LogisticRegression menegaskan pola ini melalui skor training/test dan plot koefisien.

Untuk klasifikasi multikelas, dataset sintesis make_blobs dengan tiga kelas digunakan untuk menunjukkan bahwa LinearSVC yang secara alami hanya mendukung klasifikasi biner dapat diperluas ke multikelas melalui pendekatan one-vs-rest: satu model garis dilatih untuk setiap kelas melawan kelas lainnya, dan prediksi akhir ditentukan oleh garis yang menghasilkan skor tertinggi.

2.5 Naive Bayes Classifier

Naive Bayes adalah keluarga model klasifikasi yang sangat cepat karena hanya mempelajari statistik setiap fitur per kelas secara terpisah. Pendekatan ini disebut naive karena mengasumsikan independensi antar fitur, yang sering kali tidak terpenuhi di dunia nyata. BernoulliNB cocok untuk fitur biner, GaussianNB untuk fitur kontinu, dan MultinomialNB untuk data dengan fitur non-negatif seperti frekuensi kata dalam text classification.

Pada eksperimen menggunakan dataset biner buatan tangan, BernoulliNB menghitung berapa kali setiap fitur bernilai 1 pada setiap kelas. Informasi ini digunakan untuk mengestimasi probabilitas kondisional $P(\text{fitur} \mid \text{kelas})$ yang kemudian digabungkan menggunakan teorema Bayes untuk menghasilkan prediksi kelas. Karena hanya menghitung statistik sederhana per fitur, Naive Bayes sangat cepat dilatih dan menjadi baseline yang baik untuk dataset besar atau berdimensi sangat tinggi.

2.6 Decision Trees (Pohon Keputusan)

Decision Tree membentuk model prediksi berupa struktur pohon dengan menentukan aturan jika-maka berdasarkan nilai fitur. Setiap node internal pohon berisi satu pertanyaan tentang fitur tertentu, dan setiap cabang merepresentasikan jawaban dari pertanyaan tersebut. Daun pohon berisi prediksi kelas atau nilai target.

DecisionTreeClassifier tanpa batas kedalaman pada dataset Breast Cancer mencapai akurasi training 100% tetapi akurasi test lebih rendah, tanda overfitting yang parah karena pohon tumbuh hingga setiap daun murni sempurna dan menghafal data latih. Ketika kedalaman dibatasi ($\text{max_depth}=4$), akurasi training sedikit menurun namun akurasi test justru meningkat, menunjukkan generalisasi yang lebih baik.

Feature importance menunjukkan seberapa penting setiap fitur dalam membuat prediksi pada pohon keputusan. Fitur dengan nilai importance tinggi digunakan lebih sering dan lebih dekat ke akar pohon. Pada dataset Cancer, beberapa fitur seperti worst radius dan worst concave points memiliki importance yang sangat tinggi, sedangkan banyak fitur lainnya hampir tidak digunakan.

Salah satu keterbatasan Decision Tree untuk regresi adalah ketidakmampuannya berekstrapolasi ke luar rentang nilai data latih. Studi kasus harga RAM membandingkan DecisionTreeRegressor dan LinearRegression dalam memprediksi harga setelah tahun 2000 berdasarkan data sebelum tahun 2000: prediksi pohon menjadi datar setelah tahun 2000 karena hanya dapat memprediksi nilai yang sudah ada pada data latih, sedangkan Linear Regression tetap dapat mengikuti tren karena persamaan linearnya terus berlanjut.

2.7 Random Forest

Random Forest adalah metode ensemble yang menggabungkan banyak Decision Tree yang dibuat secara independen dengan cara bootstrap aggregation (bagging). Setiap pohon dilatih pada subset acak dari data latih dan hanya mempertimbangkan subset acak fitur pada setiap split. Prediksi akhir adalah mayoritas suara (klasifikasi) atau rata-rata (regresi) dari semua pohon.

Pada dataset make_moons, RandomForestClassifier dengan 5 pohon divisualisasikan bersama batas keputusan dari 5 pohon individual (acak) dan hasil gabungannya. Setiap pohon memiliki batas keputusan yang berbeda-beda dan terlihat kasar, tetapi hasil gabungan dari semua pohon menghasilkan batas keputusan yang jauh lebih halus dan mulus, inilah keunggulan ensemble method dibandingkan satu model tunggal.

Pada dataset Breast Cancer, Random Forest dengan 100 pohon menghasilkan akurasi yang umumnya lebih tinggi dan lebih stabil dibandingkan satu Decision Tree tunggal. Feature importance dari Random Forest lebih menyebar dibandingkan satu pohon tunggal, karena dengan 100 pohon setiap fitur mendapat kesempatan untuk berkontribusi pada setidaknya beberapa pohon, menghasilkan estimasi importance yang lebih stabil dan reliabel. Fitur worst radius dan worst concave points tetap menjadi fitur terpenting.

2.8 Gradient Boosting

Gradient Boosting adalah metode ensemble yang membangun pohon secara berurutan, di mana setiap pohon baru berusaha memperbaiki kesalahan dari pohon sebelumnya. Berbeda dengan Random Forest yang membangun pohon secara paralel dan independen, Gradient Boosting membangun pohon secara iteratif dengan mengurangi residual error secara bertahap.

Dua strategi regularisasi diuji pada dataset Breast Cancer: membatasi `max_depth=1` (menggunakan decision stumps) untuk mengurangi kompleksitas tiap pohon individual, dan menurunkan `learning_rate=0,01` untuk mengurangi kontribusi tiap pohon terhadap prediksi akhir. Keduanya menurunkan akurasi training namun bertujuan meningkatkan generalisasi pada data test.

Feature importance dari Gradient Boosting menunjukkan distribusi yang lebih terpusat dibandingkan Random Forest, beberapa fitur diabaikan sepenuhnya (nilai 0). Ini terjadi karena Gradient Boosting lebih selektif dalam memilih fitur yang paling informatif untuk memperbaiki error residual. Parameter penting Gradient Boosting meliputi `n_estimators` (jumlah pohon), `learning_rate` (seberapa besar kontribusi setiap pohon), dan `max_depth` (kedalaman maksimal pohon).

2.9 Support Vector Machine (SVM)

Support Vector Machine (SVM) adalah algoritma klasifikasi yang berusaha menemukan hyperplane dengan margin maksimum yang memisahkan dua kelas. Titik-titik data yang paling dekat dengan hyperplane disebut support vectors. SVM dapat bekerja pada ruang dimensi tinggi dan menggunakan kernel trick untuk memisahkan data yang tidak linearly separable.

Menggunakan dataset `make_blobs` empat klaster yang label kelasnya dibuat non-linearly separable, ditunjukkan bahwa `LinearSVC` gagal memisahkan data dengan baik karena batas keputusannya berupa garis lurus. Dengan menambahkan fitur baru berupa kuadrat dari salah satu fitur dan memvisualisasikannya dalam ruang 3D, data yang semula tidak terpisahkan secara linear di 2D menjadi terpisahkan oleh sebuah hyperplane datar di ruang 3D, ini adalah intuisi dasar di balik kernel trick.

Grid kombinasi parameter `C` (kekuatan regularisasi) dan `gamma` (lebar kernel RBF) menunjukkan bahwa `gamma` kecil menghasilkan batas keputusan yang mulus (mempertimbangkan banyak titik), `gamma` besar menghasilkan batas yang sangat tidak teratur. `C` kecil memberikan lebih banyak toleransi terhadap misklasifikasi, `C` besar berusaha mengklasifikasikan semua titik dengan benar meski menghasilkan batas yang lebih kompleks.

SVM sangat sensitif terhadap skala fitur. Pada dataset Cancer, fitur-fitur memiliki rentang nilai yang sangat berbeda-beda (dalam skala logaritmik). Tanpa penskalaan, SVM mengalami performa yang buruk (akurasi test hanya sekitar 0,63). Setelah penskalaan min-max, akurasi melonjak drastis karena semua fitur berada dalam rentang $[0, 1]$.

2.10 Neural Networks (MLPClassifier)

Neural Network atau Multilayer Perceptron (MLP) adalah algoritma yang terinspirasi dari cara kerja neuron di otak manusia. MLP terdiri dari beberapa lapisan: input layer, satu atau

lebih hidden layers, dan output layer. Setiap lapisan terhubung penuh ke lapisan berikutnya, dan setiap koneksi memiliki bobot yang dipelajari selama training.

Fungsi aktivasi membuat neural network mampu mempelajari pola non-linear. ReLU (Rectified Linear Unit) adalah fungsi $\max(0, x)$ yang sangat populer karena komputasinya sederhana dan efektif. Tanh (hyperbolic tangent) menghasilkan output antara -1 dan 1, lebih smooth dibandingkan ReLU.

Perbandingan berbagai konfigurasi MLP pada dataset `make_moons` menunjukkan bahwa jumlah unit hidden layer, jumlah layer, fungsi aktivasi, dan parameter regularisasi α semuanya memengaruhi kompleksitas batas keputusan. MLP dengan aktivasi tanh dan dua hidden layer menghasilkan batas yang lebih mulus dibandingkan ReLU. α yang lebih besar memberikan regularisasi lebih kuat dan batas yang lebih sederhana. Inisialisasi bobot acak yang berbeda juga menunjukkan bahwa MLP dapat menghasilkan batas keputusan yang cukup bervariasi antar pelatihan meski konfigurasi sama.

Sama seperti SVM, MLP default pada dataset Breast Cancer memberikan performa yang kurang optimal karena rentang nilai fitur yang sangat bervariasi. Setelah fitur distandarisasi (dikurangi rata-rata, dibagi standar deviasi dari data latih), performa meningkat signifikan. Heatmap bobot antara layer input dan hidden layer pertama memberikan gambaran tentang bagaimana neural network melihat data: warna merah menunjukkan bobot positif (fitur meningkatkan aktivasi neuron), warna biru menunjukkan bobot negatif.

2.11 Ketidakpastian dalam Prediksi

Bagian ini membahas cara menginterpretasikan kepercayaan diri (confidence) model dalam prediksinya. Scikit-learn menyediakan dua cara utama: `decision_function` (mengembalikan skor raw per kelas) dan `predict_proba` (mengembalikan probabilitas yang telah dinormalisasi antara 0 dan 1 untuk setiap kelas).

Pada kasus biner menggunakan `GradientBoostingClassifier` pada dataset `make_circles`, `decision_function` menghasilkan nilai kontinu yang sulit diinterpretasikan secara langsung karena skalanya bervariasi antar model dan dataset, meskipun thresholding `decision_function > 0` menghasilkan prediksi yang identik dengan `predict()`. `Predict_proba` lebih mudah dipahami karena nilainya selalu antara 0 dan 1 dan jumlahnya selalu 1 untuk setiap sampel. Gradasi warna pada `predict_proba` lebih jelas menunjukkan area ketidakpastian model (probabilitas mendekati 0,5 berarti model tidak yakin).

Pada kasus multikelas menggunakan dataset Iris, `decision_function` menghasilkan satu skor per kelas untuk tiap sampel, dan kelas dengan skor tertinggi (`argmax`) menjadi prediksi akhir. Eksperimen dengan LogisticRegression yang dilatih menggunakan nama kelas string

menunjukkan bahwa urutan `classes_` (diurutkan secara alfabetis) harus digunakan untuk memetakan `argmax` dari `decision_function` kembali menjadi label kelas yang benar, poin penting yang sering menjadi sumber kesalahan interpretasi bila diabaikan.